

- * replace: 'N' → ' ', 'FRAGMENT_...' → ' ' ✓
- * case = ' '.join(case.split()) ✓
- * segment by sentences not words/characters ✓
- * sum scores for each candidate ✓ ⇒ done but with macro-BM25 (arguably better)
- * only return candidates whom - $\text{score} > \max(\text{score}) \cdot p$
p is threshold percentage

- * to prevent numbers from becoming too dominant we can use IDF capping or replace unuseful numbers with NUM or DATE (implemented only 2) \
- we also can simply discard any term whom appears in less than 3 docs X
- * removing stop words X → not yet

suggestion: in the macro-BM25 implementation for each hit per q we track the sum of scores it got across all query paragraphs. we ignore the case where a query paragraph is scoring everyone highly. how about for each query para we give it "money" and it divides it as it would like on all the hits?

- * BM25 params understanding & correct setup. ✓
- * if we get score 10 and score 5 does that mean case A is x2 tolerant from 1 how should we translate it into probabilities?
- * if we apply the suggestion would the increase of para-size help the results?
Gemini suggests to find the sweet spot ⇒ grid search
- ** note that we still haven't handled the duplicate cases problem
- ** parsing, tokenization and clean up is slightly scattered, in that case how to handle queries
parsing:
- * problem with suggestion: we might reward paragraphs who we have low confidence in them, if we query A and get (B, 1) ... we give B many points without if we are not confident in it!

refined suggestion

- 1) measure paragraph importance ⇒ avg IDF?
- 2) distribute importance on the hit set
but only for confident matches!